

ByteFUSE分布式文件系统的演进与落地

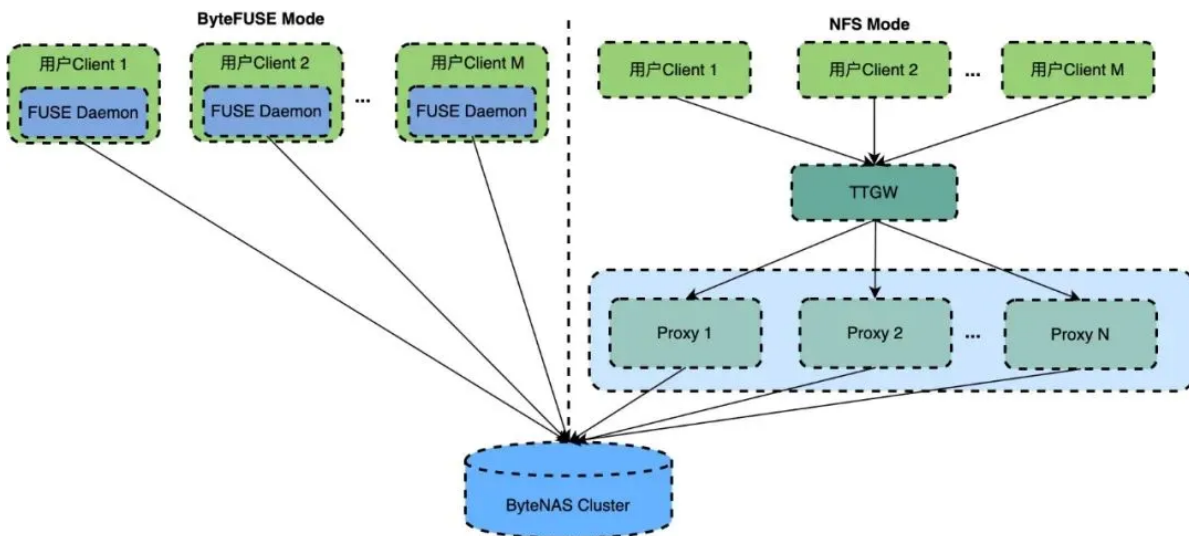
导语：ByteFUSE是ByteNAS团队和STE团队合作研发的一个项目，因其具有高可靠性、极致的性能、兼容Posix语义以及支持丰富的使用场景等优点而被业务广泛使用。目前承接了在线业务ES，AI训练业务，系统盘业务，数据库备份业务，消息队列业务，符号表业务以及编译业务等，字节内部部署机器和日常挂载点均已达到万级规模，总吞吐近百GB/s，容量十几PB，其性能与稳定性能够满足业务需求。

背景

ByteNAS是一款全自研、高性能、高扩展，多写多读、低时延并且完全兼容Posix语义的分布式文件系统，目前支撑了字节内部AI训练，数据库备份，在线ES等多个关键业务，也是未来云上NAS主打的产品形态。早期ByteNAS对外提供服务使用的是NFS协议，其依赖**TTGW四层负载均衡器**将外部流量以TCP连接的粒度均衡到连接的多台Proxy，用户使用TTGW提供的VIP并进行挂载即可与多台Proxy中一台进行通信。如果当前通信的Proxy因为机器宕机等原因挂掉后，**TTGW内部探测心跳超时会触发Failover机制**，自动将来自该Client的请求Redirect到新的活着的Proxy，该机制对客户端是完全透明的。但是使用TTGW具有以下缺点：

- 无法支持大吞吐场景：用户的吞吐不仅受限于TTGW集群本身吞吐的限制，而且受限于NFS协议单次读写1MB的限制。另外NFS是单TCP连接，同时内核slot并发请求也有限制，这会导致吞吐受限以及元数据和数据相互影响
- 额外的网络延迟：用户访问ByteNAS多两跳网络（用户侧**NFS Client -> TTGW -> Proxy -> ByteNAS**）
- 额外的机器成本：需要TTGW以及Proxy等机器资源
- 定制化业务需求以及性能优化比较困难：受限于内核NFS Client，NFS协议以及TTGW的影响，其定制化需求以及性能优化比较困难

为了解决以上问题，ByteFUSE应运而生。ByteFUSE是一套基于用户态文件系统（FUSE）框架接入ByteNAS的解决方案，通过ByteNAS SDK直连ByteNAS集群，不仅满足了低延迟的目标，同时也解决了协议吞吐受限的问题。除此之外，由于部分文件系统逻辑上移到了用户态，对于问题排查，功能扩展以及性能优化都会变得非常方便。用户使用ByteFUSE和NFS两种协议访问ByteNAS的流程如下图所示：



目标

- 高性能、低延迟，对业务友好的架构模型设计

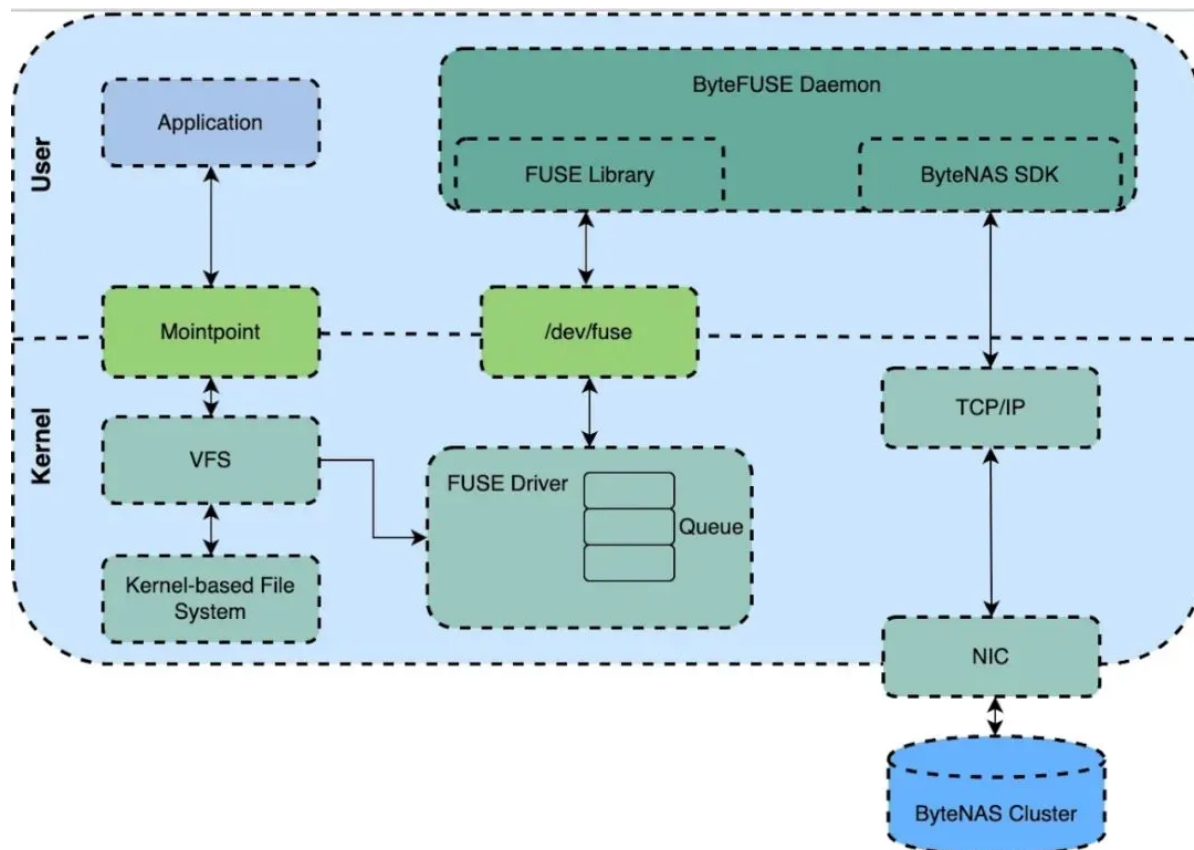
- 完全兼容Posix语义
- 支持一写多读/多写多读
- 自研以及可维护性强，提供定制化特性能力支持

演进路线

1. ByteFUSE 1.0 — 基础功能完备，云原生部署支持

通过原生FUSE接入ByteNAS

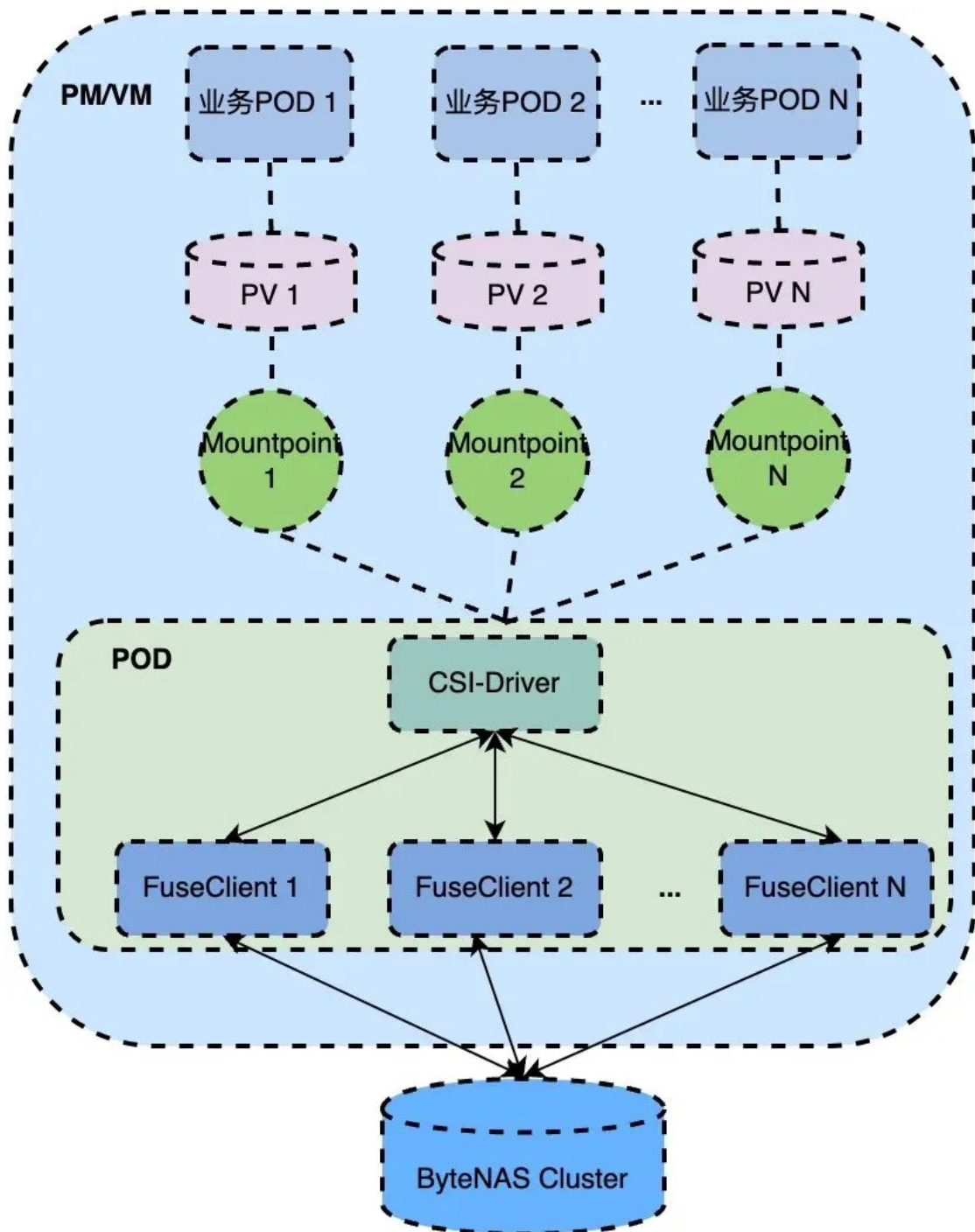
原生FUSE对接ByteNAS的整体架构图如下所示：



ByteFUSE Daemon: 集成了ByteNAS SDK的FUSE Daemon，用户的文件系统请求会通过FUSE协议转发给ByteFUSE Daemon，然后，通过ByteNAS SDK被转发到后端存储集群。

云原生部署支持

ByteFUSE基于K8S CSI接口规范 [1] 开发了CSI插件，以支持在K8S集群中使用ByteFUSE访问ByteNAS集群，其架构如下图所示：



- CSI-Driver: ByteFUSE的云原生架构目前只支持静态卷，Mount/Umount操作会在CSI-Driver中启动/销毁FUSE Client，CSI-Driver会记录每个挂载点的状态，当CSI-Driver异常退出重启时会recover所有挂载点来保证高可用性。
- FUSE Client: 即上面提到的ByteFUSE Daemon，在1.0架构下，针对每个挂载点，CSI-Driver都会启动一个FUSE Client来提供服务。

2. ByteFUSE 2.0 — 云原生架构升级，一致性、可用性和可运维性提升

业务需求和挑战

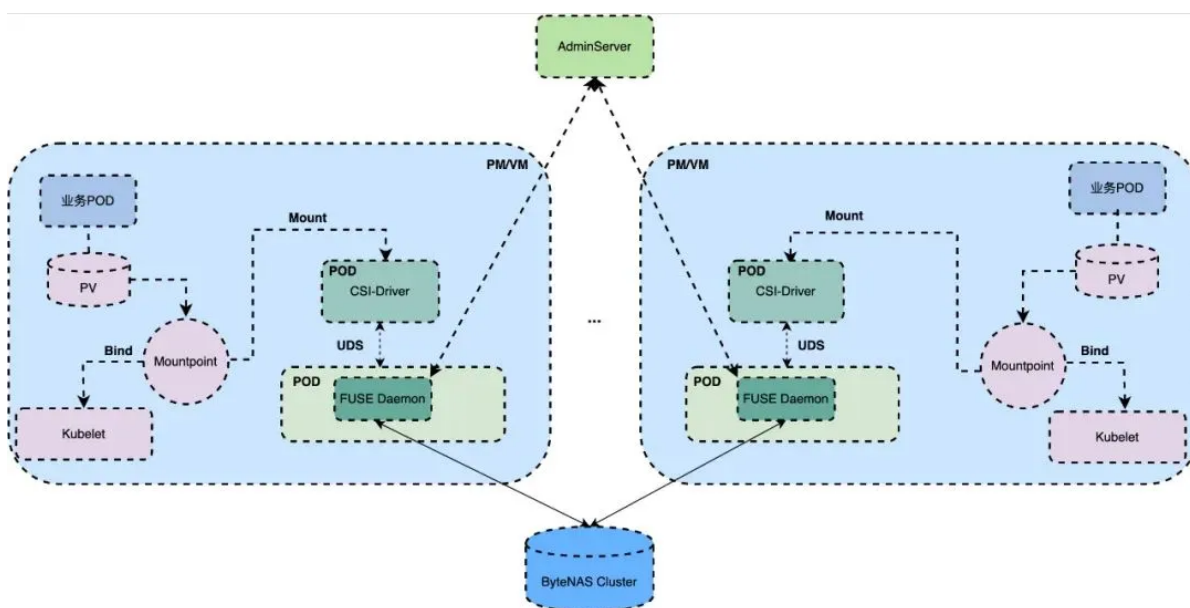
- **FUSE Client资源占用不可控以及无法复用**: 多FUSE Client模式下，一个挂载点对应一个FUSE Client进程，FUSE Client的资源占用与挂载点个数强相关，这导致FUSE Client资源占用不可控。

- **FUSE Client与CSI-Driver强耦合导致CSI-Driver无法平滑升级**：FUSE Client进程的生命周期与CSI-Driver关联，当需要升级CSI时，FUSE Client也需要跟随重建，导致业务I/O也会受影响，同时，这个影响时长与CSI-Driver的升级时长（秒级）强相关。
- **部分业务希望在Kata容器场景接入ByteFUSE**：云原生场景下，有部分业务会以Kata容器的方式来运行，为了满足这部分业务接入ByteFUSE的需求，CSI-Driver需要支持kata这种容器运行时，即在kata虚拟机内能够通过ByteFUSE访问ByteNAS服务。
- **原生FUSE一致性模型无法满足某些业务需求**：某些业务是典型的一写多读场景，对读写吞吐，数据可见性以及尾延迟的要求极高，但原生FUSE在开启内核缓存的情况下，无法提供像CTO (Close-to-Open) 这样的一致性模型。
- **原生FUSE可用性/可运维能力较弱，无法适用于大规模生产环境**：原生FUSE对高可用、热升级等能力的支持较弱，当出现FUSE进程crash或者内核模块有bug需要升级等情况时，往往需要知会业务重启Pod、甚至重启整个物理节点，这对于大部分业务都是不可接受的。

云原生架构升级

FUSE Client架构升级：单Daemon化

针对上述业务需求和挑战，我们对架构进行了升级，支持了单FUSE Daemon模式来解决资源不可控以及资源无法复用的问题，采用FUSE Daemon和CSI-Driver的分离解决CSI-Driver无法平滑升级的问题，其架构如下图所示：

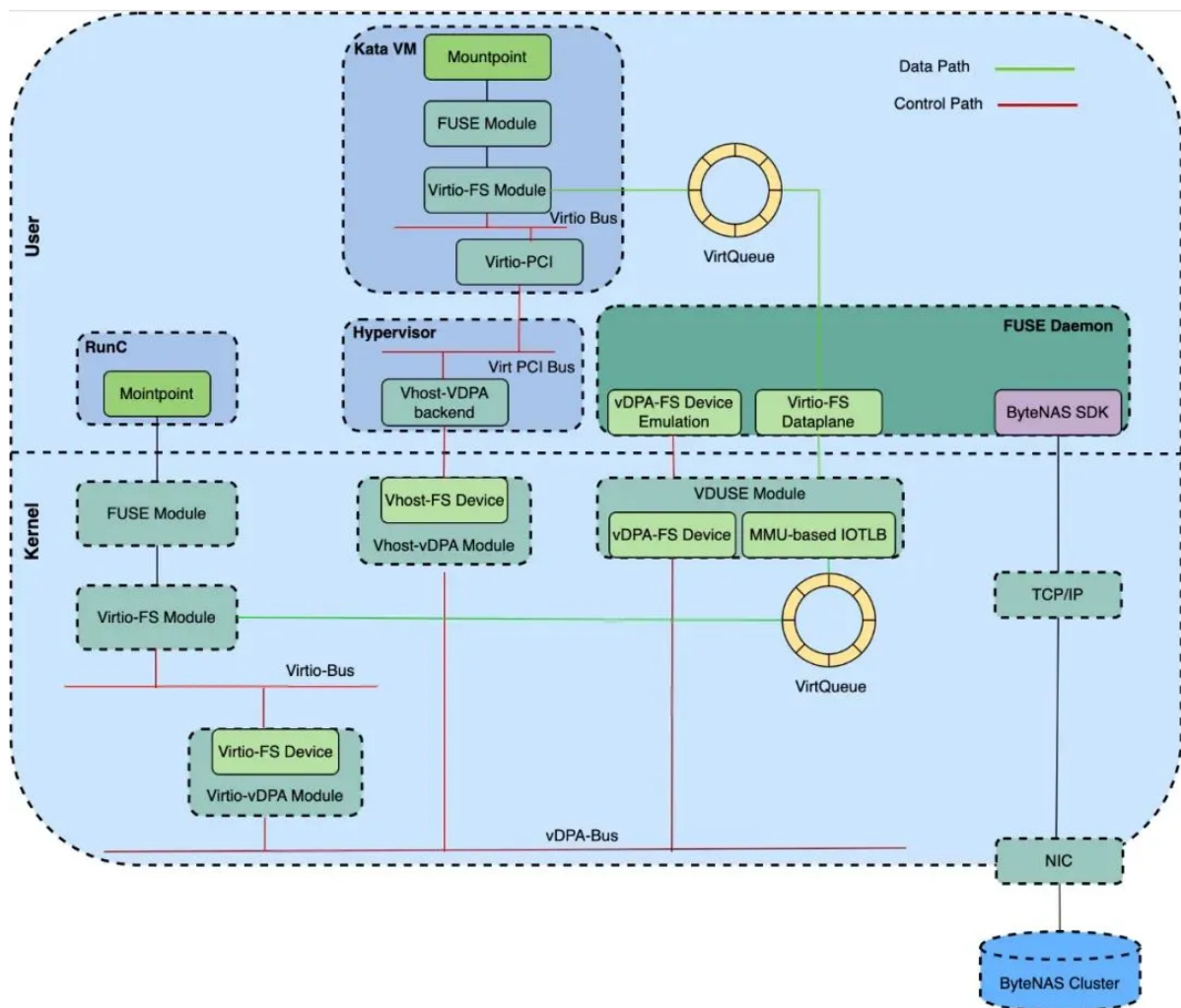


AdminServer：监控Mountpoint/FUSE Daemon状态，升级FUSE Daemon以及统计集群信息等。

FUSE Daemon：管理ByteNAS集群所有的挂载点以及处理读写请求，重启后recover所有的挂载点，恢复时间为ms级别。

Kata Containers 场景支持

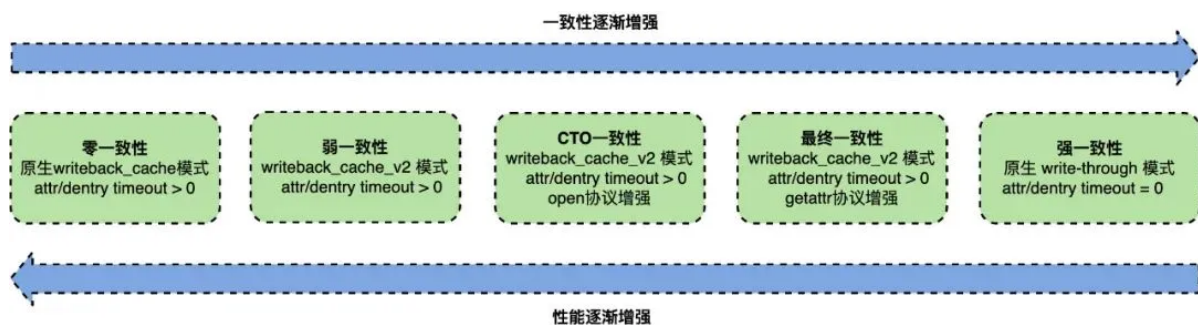
为了提供Kata场景的支持，同时，解决原生FUSE的高可用和性能可扩展性问题，我们在2.0架构中引入了VDUSE[2]这个字节自主研发的技术框架来实现ByteFUSE Daemon。VDUSE利用了virtio这套成熟的软件框架，使ByteFUSE Daemon能够同时支持从虚拟机或者宿主机（容器）挂载。同时，相较于传统的FUSE框架，基于VDUSE实现的FUSE Daemon不再依赖/dev/fuse这个字符设备，而是通过共享内存机制来和内核通信，这种方式一方面对后续的性能优化大有裨益，另一方面也很好地解决了Crash Recovery问题。



一致性、可用性和可运维性提升

一致性模型增强

性能和一致性是分布式系统设计中的一对根本性矛盾——保持一致性意味着更多节点的通信，而更多节点的通信意味着性能的下降。为了满足相关业务需求，我们在FUSE原生缓存模式的基础上不断的取舍性能与一致性，实现了 FUSE CTO (Close-to-Open) 一致性模型 [4]，并将这些一致性模型根据不同配置抽象成以下五种：

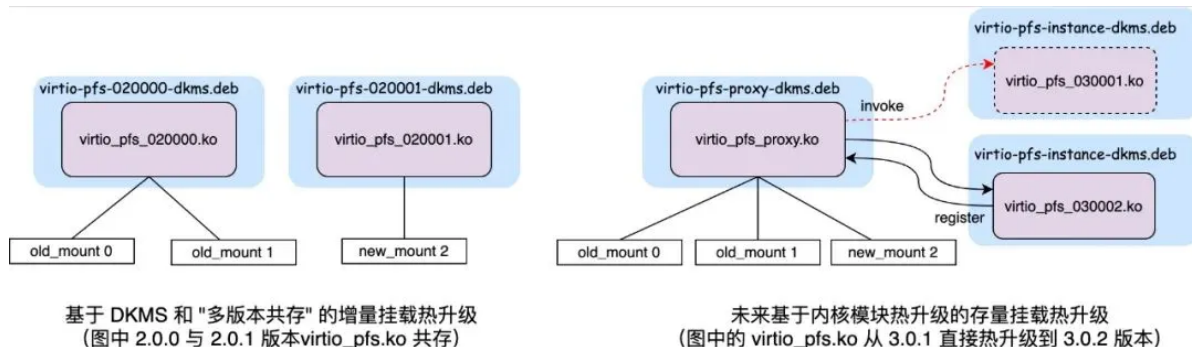


Daemon高可用

由于ByteFUSE 2.0架构引入了VDUSE [2]这个技术框架，因此支持使用基于共享内存的Virtio协议作为传输层，Virtio协议内置的inflight I/O追踪特性可以将 ByteFUSE 正在处理的请求实时持久化，并在 ByteFUSE 恢复时重新处理未完成请求，这弥补了原生 libfuse 中使用字符设备 /dev/fuse 作为传输层时状态保存功能的缺失。基于该inflight I/O 追踪特性，ByteFUSE 进一步考虑了文件系统状态在恢复前后的一致性和幂等性，实现了用户无感的崩溃恢复 [3]，同时基于崩溃恢复实现了Daeamon的热升级。

内核模块热升级

ByteFUSE 在使用定制化内核模块来获得更好的性能、可用性和一致性的同时，也对这些定制化内核模块的升级维护提出挑战。为了解决二进制内核模块无法随内核升级的问题，我们通过 DKMS 来部署定制化内核模块，让内核模块随内核升级而自动重新编译部署。为了解决内核模块自身热升级的问题，我们通过将内核模块所导出的符号名或设备号与版本号绑定的方式实现了同一内核模块的多版本共存。新的 ByteFUSE 挂载将自动使用新的内核模块；旧的 ByteFUSE 挂载延续使用旧内核模块。



目前，通过上述 DKMS 技术及“多版本共存”技术，我们将ByteFUSE内核模块的升级与内核及ByteFUSE Daemon进行了解耦；未来，我们将进一步实现ByteFUSE内核模块热升级功能，以支持线上运行的存量ByteFUSE卷的热升级功能。

3. ByteFUSE 3.0 — 极致性能优化，打造业界一流的高性能文件存储系统

业务需求和挑战

大模型训练场景对存储系统的性能需求

大模型训练场景下，训练巨量模型需要巨大的算力，但随着数据集和模型规模不断增加，应用程序载入数据所花费的时间变得越长，进而影响了应用程序的性能，**缓慢的 I/O 严重拖累GPU 的强大算力**。于此同时，模型的评估 & 部署需要并行读取大量模型，要求存储能够提供超高吞吐。

云原生高密部署的场景，需要进一步降低资源占用开销

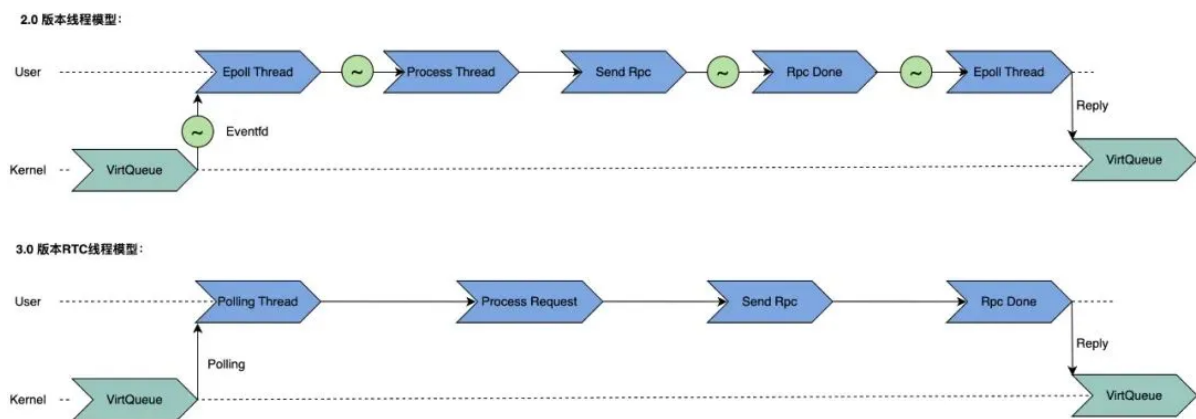
云原生高密部署场景下，随着ByteFUSE卷的数量级增加，对ByteFUSE单机侧的资源(CPU & Memory)占用及隔离提出了新的要求。

极致性能优化

ByteFUSE 3.0从线程模型，数据拷贝，内核侧以及协议栈进行了全链路的性能优化，性能提高2.5倍，2个core即可打满百Gb网卡。其优化方向如下所示：

Run-to-Completion线程模型

2.0 版本的一次Read/Write请求会有4次线程切换，接入Run-to-Completion (RTC) 能够节省这四次线程切换带来的开销。为了做到Run-to-Completion，我们对ByteFUSE和ByteNAS SDK进行了【**shared-nothing**】的设计和锁的非阻塞化改造，其目的是保证RTC线程不会被阻塞，避免影响请求的延迟。

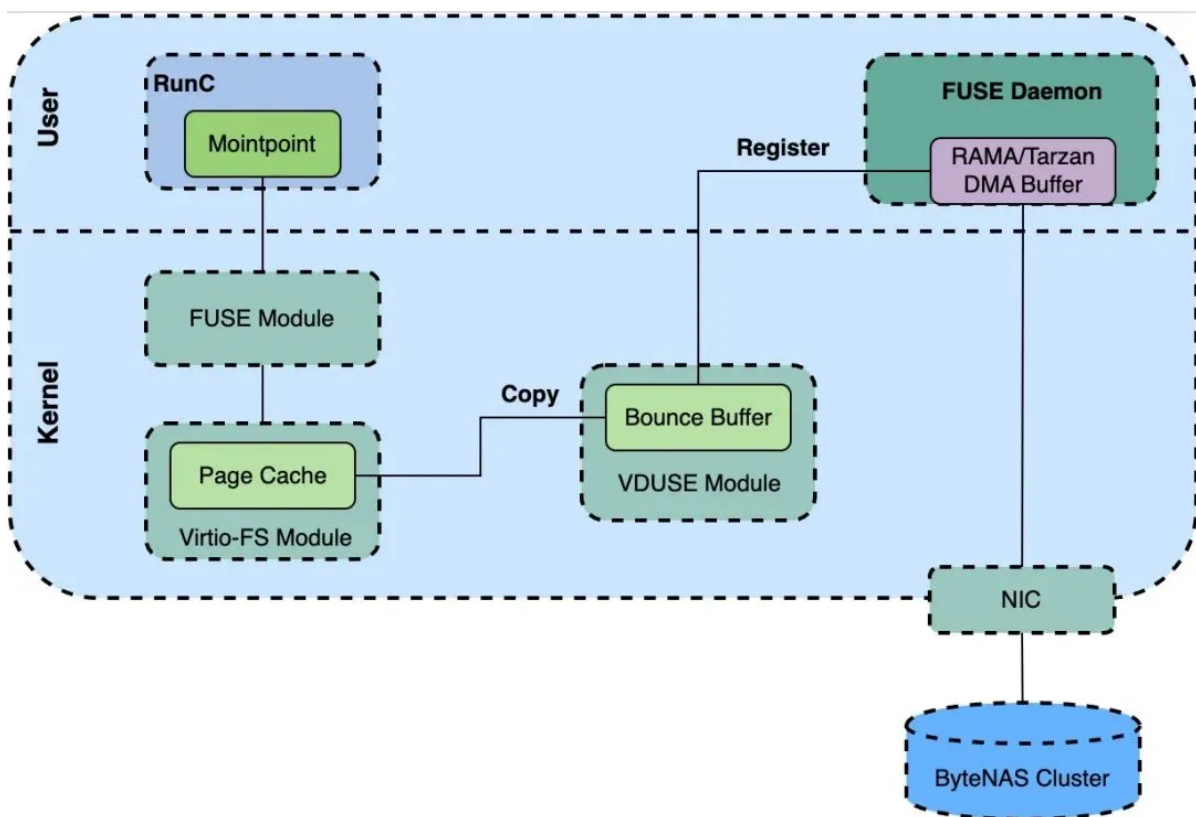


RDMA & 用户态协议栈

3.0架构相较于2.0，在网络传输这块也做了较大的改进，主要体现在引入了RDMA和用户态协议栈（Tarzan）来替换传统的内核TCP/IP协议栈，相较于内核TCP/IP协议栈，RDMA/Tarzan能够节省用户态与内核态的切换和数据拷贝带来的延迟，并进一步降低CPU占用。

全链路零拷贝

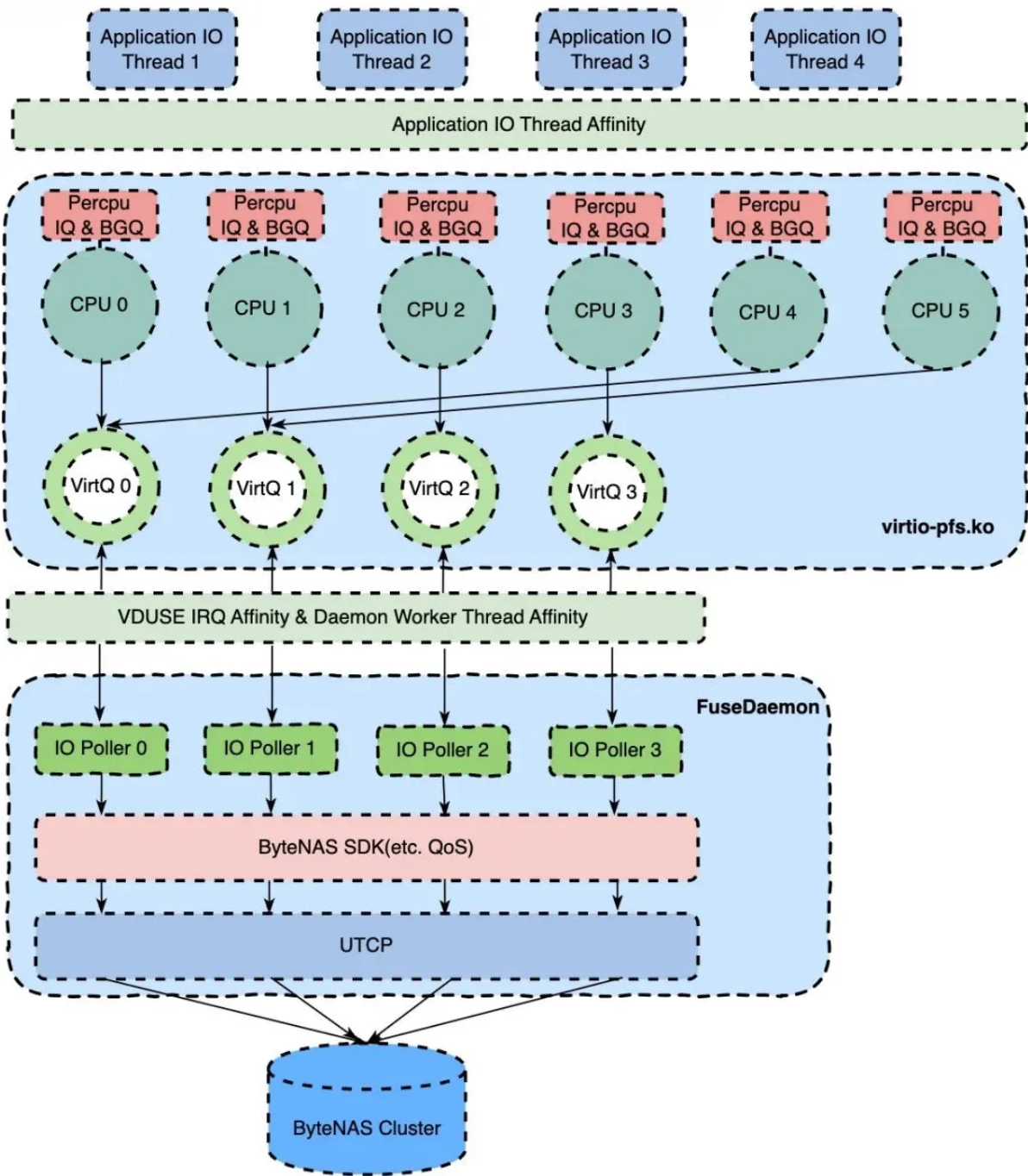
引入**RDMA/Tarzan**之后，ByteFUSE在网络传输这块的拷贝被成功消除了，但FUSE接入这块，仍然存在Page Cache到Bounce Buffer，Bounce Buffer到RDMA/Tarzan DMA Buffer两次拷贝。为了降低这部分的拷贝开销（经统计1M数据的拷贝消耗100us左右），3.0架构引入了VDUSE umem [5] 特性，通过将RDMA/Tarzan DMA Buffer注册给VDUSE内核模块，减少了其中的一次拷贝。未来，我们还会进一步实现FUSE PageCache Extension特性，来达成**全链路零拷贝**这个优化目标。



FUSE内核优化

(1) 多队列

在原生FUSE/virtiofs内核模块中，FUSE 请求的处理路径有很多单队列设计：如每个 FUSE 挂载只有一个 IQ (input queue)、一个 BGQ (background queue)、virtiofs 设备使用单队列模型发送 FUSE 请求等。为了减少单队列模型带来的锁竞争、提高可扩展性，我们在 FUSE/virtiofs 的请求路径中支持了 per-cpu 的 FUSE 请求队列和可配置的 virtiofs virtqueue 数量。基于 FUSE 多队列特性的支持，ByteFUSE 可以根据不同部署环境配置不同的 CPU 亲和性和策略来减少核间通信或平衡核间负载。ByteFUSE 工作线程也可以打开 FUSE 多队列特性所提供的负载均衡调度来缓解核间请求不均情况下的局部请求排队现象。



(2) huge块支持

为了满足高吞吐场景的性能需求，ByteFUSE 3.0 版本支持定制化的FUSE内核模块参数。Linux 内核原生的FUSE 模块中存在一些对数据传输的硬编码，如单次最大数据传输单元为 1 MB，单次最大目录树读取单元为 4 KB。而在ByteFUSE内核模块中，我们将单次最大数据传输上调为 8 MB， 单次最大目录读取单元上调为 32 KB。在数据库备份场景下，将单次写下刷改成8MB，单机吞吐能提高约20%。

演进收益

收益总览

1.0 -> 2.0

- **降低资源占用，便于资源控制**

单FUSE Daemon和多FUSE Client相比，多个挂载点之间的线程、内存、连接等资源可以复用，可以有效降低资源占用。除此之外，将FUSE Daemon单独运行于Pod内能够更好地适应Kubernetes生态，保证其在 Kubernetes 的管控内，用户可以直接在集群中观察到FUSE Daemon的Pod，可观测性强。

- **CSI-Driver与FUSE Daemon解耦合**

CSI-Driver与FUSE Daemon作为两个独立部署的服务，其部署、升级都可以独立进行而不影响彼此，进一步降低了运维工作对业务的影响。除此之外，我们支持POD内热升级FUSE Daemon，整个升级对业务是无感的。

- **支持内核模块热升级**

可以在业务无感的情况下，支持对ByteFUSE增量卷进行热升级，修复内核模块的已知bug，降低线上风险。

- **支持统一的监控以及管控平台，方便可视化管理**

AdminServer监控一个Region内所有FUSE Daemon & 挂载点的状态，支持远程恢复异常挂载点，支持Pod内热升级FUSE Daemon，支持远程挂载点异常探测以及报警等。

2.0 -> 3.0

整个架构实现了Run-to-Complete的线程模型，减少了锁以及上下文切换带来的性能损耗。除此之外，我们将内核态TCP换成用户态TCP，bypass内核以及采用内存注册到内核的方式实现全链路zero-copy进一步的提升性能。对于1MB的写请求，FUSE Daemon侧可以节省百us。

性能对比

FUSE Daemon 机器规格：

- CPU: 32物理核，64逻辑核
- 内存: 251.27GB
- NIC: 100Gbps

元数据性能对比

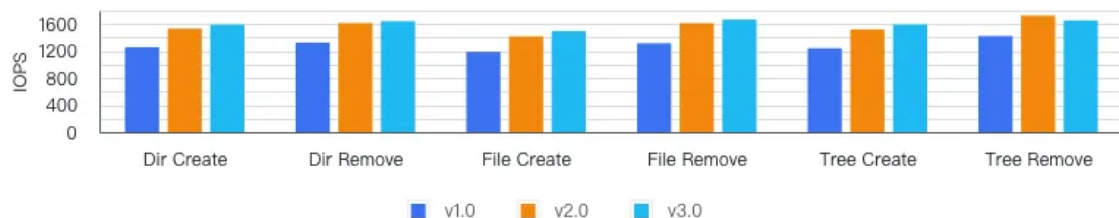
使用mdtest进行性能测试，测试命令

-

```
mdtest '-d' '/mnt/mdtest/' '-b' '6' '-I' '8' '-z' '4' '-i' '30'
```

，性能差异如下所示：

mdtest 基准测试



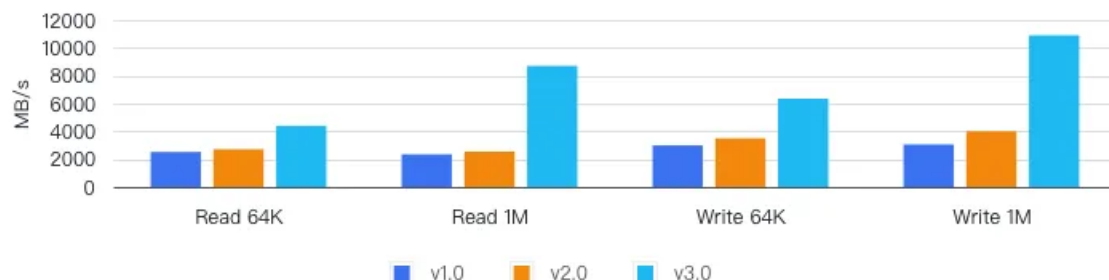
结论

3.0架构相比1.0架构元数据性能大约提高了25%左右。

数据性能对比

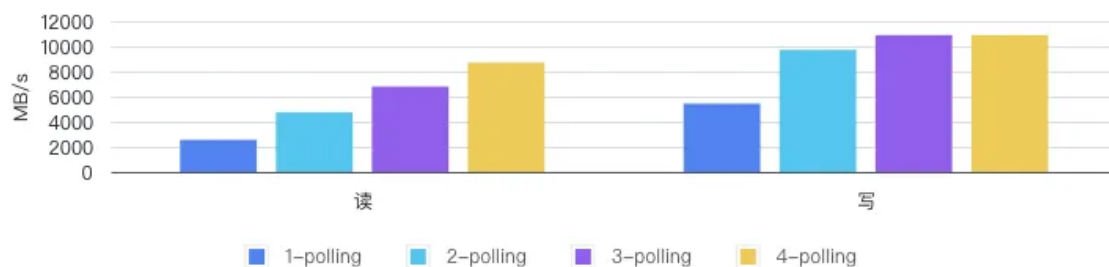
FIO采用4线程，其性能如下图所示：

FIO DirectIO 读写性能



此外，测试ByteFUSE 3.0 polling线程个数对性能的影响。对于写，2个polling线程基本打满百G网卡，而对于读，则需要4个polling线程(比写操作多一次数据拷贝)。未来，我们将改造用户态协议栈Tarzan，节省读的一次数据拷贝，做到读写zero-copy。

FIO DirectIO 读写性能



业务落地

ES存算分离架构场景的落地

场景描述

ES的Shared Storage架构，让ES 的多个分片副本使用同一份数据，来解决在 Shared Nothing 架构下的扩容生效缓慢，分片迁移慢，搜索打分振荡以及存储成本高等问题。底层存储使用ByteNAS来共享主副分片的数据以及使用ByteFUSE作为接入协议来满足高性能，高吞吐以及低延迟的要求。

收益

ES存算分离架构的落地，每年节省存储成本近千万

AI训练场景的落地

场景描述

AI 类 Web IDE场景下使用块存储 + NFS的方式共享根文件系统无法解决由于NFS断联进程进入D状态以及NFS断联触发内核Bug导致mount功能不可用等问题。除此之外，AI训练场景下受限负载均衡的吞吐以及NFS协议性能的影响无法满足训练任务高吞吐 & 低延迟的需求，而ByteNAS提供共享文件系统，超大吞吐以及低延迟满足模型训练。

收益

满足AI训练对高吞吐，低延迟的需求

其他业务场景的落地

受限于TTGW吞吐以及稳定性的原因，数据库备份业务，消息队列业务，符号表业务以及编译业务从NFS切换到ByteFUSE。

未来展望

ByteFUSE 3.0架构已经可以满足绝大部分业务的需求，但是，为了追求更极致的性能以及满足更多的业务场景，未来我们还有不少工作有待展开：

- ByteFUSE推广到ToB场景；满足云上业务超低延迟，超高吞吐的需求
- 支持非Posix语义；定制化接口满足上层应用的需求，譬如 IO fencing语义
- FUSE PageCache Extension；FUSE支持Page Cache用户态扩展，FUSE Daemon能够直接读写Page Cache
- 支持内核模块热升级；支持用户无感情况下，升级存量及增量ByteFUSE卷的内核模块
- 支持GPU Direct Storage[6]；数据直接在RDMA网卡和GPU之间进行传输，bypaas主机内存和CPU

参考资料

[1] <https://kubernetes-csi.github.io/docs/>

[2] <https://www.redhat.com/en/blog/introducing-vduse-software-defined-datapath-virtio>

[3] <https://juejin.cn/post/7171280231238467592>

[4] <https://lore.kernel.org/lkml/20220624055825.29183-1-zhangjiachen.jaycee@bytedance.com/>

[5] <https://lwn.net/Articles/900178/>

[6] <https://docs.nvidia.com/gpudirect-storage/overview-guide/index.html>